

AT32 PMSM FOC Incremental Encoder Demo

Introduction

This application note describes how to use a motor development board with a vector control algorithm to drive a three-phase permanent magnet synchronous motor (brushless DC motor) equipped with an incremental encoder. It covers environment setup, program documentation structure, macro definition descriptions, state machine workflow, and other aspects. Arterytek offers a variety of motor development boards. This document uses the AT-MOTOR-EVB motor development board paired with the AT32F413RCT7 sub-board as an example for explanation.

Applicable products:

Part number	AT32F402/405 series
	AT32F403A/407 series
	AT32F413 series
	AT32F415 series
	AT32F421 series
	AT32F422/426 series
	AT32F425 series
	AT32F423 series
	AT32F435/437 series
	AT32F45x series
	AT32L021 series
	AT32M412/M416 series

Contents

1	Motor control library algorithm	5
2	Environment requirements	6
	2.1 Hardware requirement.....	6
	2.2 Software requirement	9
3	PMSM motor control library documents	13
4	State machine	25
	4.1 Description of state.....	25
5	How to use demo project	28
6	Revision history.....	29

List of tables

Table 1. Motor specifications	6
Table 2. Correspondence between Flash size and ROM	9
Table 3. Description of motor control library documents	14
Table 4. Drive mode definitions	16
Table 5. Control parameter definitions	17
Table 6. Driver parameter definitions	19
Table 7. Motor parameter definitions	20
Table 8. Configurations of peripherals (AT32F413RCT7)	21
Table 9. Peripheral configuration related functions	23
Table 10. Motor control related interrupt functions (AT32F413RCT7)	24
Table 11. Initial state and state switching conditions	26
Table 12. Document revision history	29

List of figures

Figure 1. JK42BLS01-X056ED motor.....	6
Figure 2. AT-MOTOR-EVB_V1.x evaluation board.....	7
Figure 3. AT-MOTOR-EVB_V2.x evaluation board.....	8
Figure 4. AT32F413RCT7 ROM settings (Keil).....	10
Figure 5. AT32F413RCT7 ROM settings (AT32IDE)	11
Figure 6. AT32F413RCT7 ROM settings (IAR).....	12
Figure 7. Structure of motor control library documents	13
Figure 8. Structure of motor control project.....	14
Figure 9. JK42BLS01-X056ED encoder ABZ.....	21
Figure 10. State machine process flow	25

1 Motor control library algorithm

The motor control library algorithm mainly contains:

Target motor type: Three-phase permanent magnet synchronous motor (PMSM) / brushless DC motor (BLDC)

Control modes:

- FOC vector control

Three-phase PWM method:

- SVPWM

Phase current sensing mode:

- 3-shunt current sensing
- 2-shunt current sensing
- 1-shunt current sensing and reconstruction method

Rotor position detection mode:

- Photoelectric incremental encoder

Sensored FOC control mode:

- Voltage vector control
- Torque control (current vector control)
- Speed control
- Field weakening control
- Positioning control

Auto tuning functions:

- Motor winding parameters identification
- Current PI controller parameters self-tuning

2 Environment requirements

2.1 Hardware requirement

A PMSM (BLDC) motor, AT-Link or third-party debugger and a motor control board (AT-MOTOR-EVB) are required. For details about hardware configurations, please refer to *AT32_LV_Motor_Control_EVB_User_Manual* (UM0011 for EVB_V1.x or UM0014 for EVB_V2.x).

- PMSM (BLDC) motor: JK42BLS01-X056ED

Figure 1. JK42BLS01-X056ED motor



Table 1. Motor specifications

Model	JK42BLS01-X056ED
Motor poles	8
Rated voltage	DC 24V
Rated speed	4000 RPM
Rated torque	0.0716 N·m
No-load current	0.3 Amps
Output power	30 W
Line-to-line resistance	1.8Ω
No-load speed	6800 RPM
Back EMF constant	4.36 V/Krpm
Torque constant	0.042 N·m/A
Encoder resolution	1000 P/R
Insulation class	Class B
Winding type	Delta

- Debugger
- Motor control board: AT-MOTOR-EVB V1.x or AT-MOTOR-EVB V2.x.

Figure 2. AT-MOTOR-EVB_V1.x evaluation board

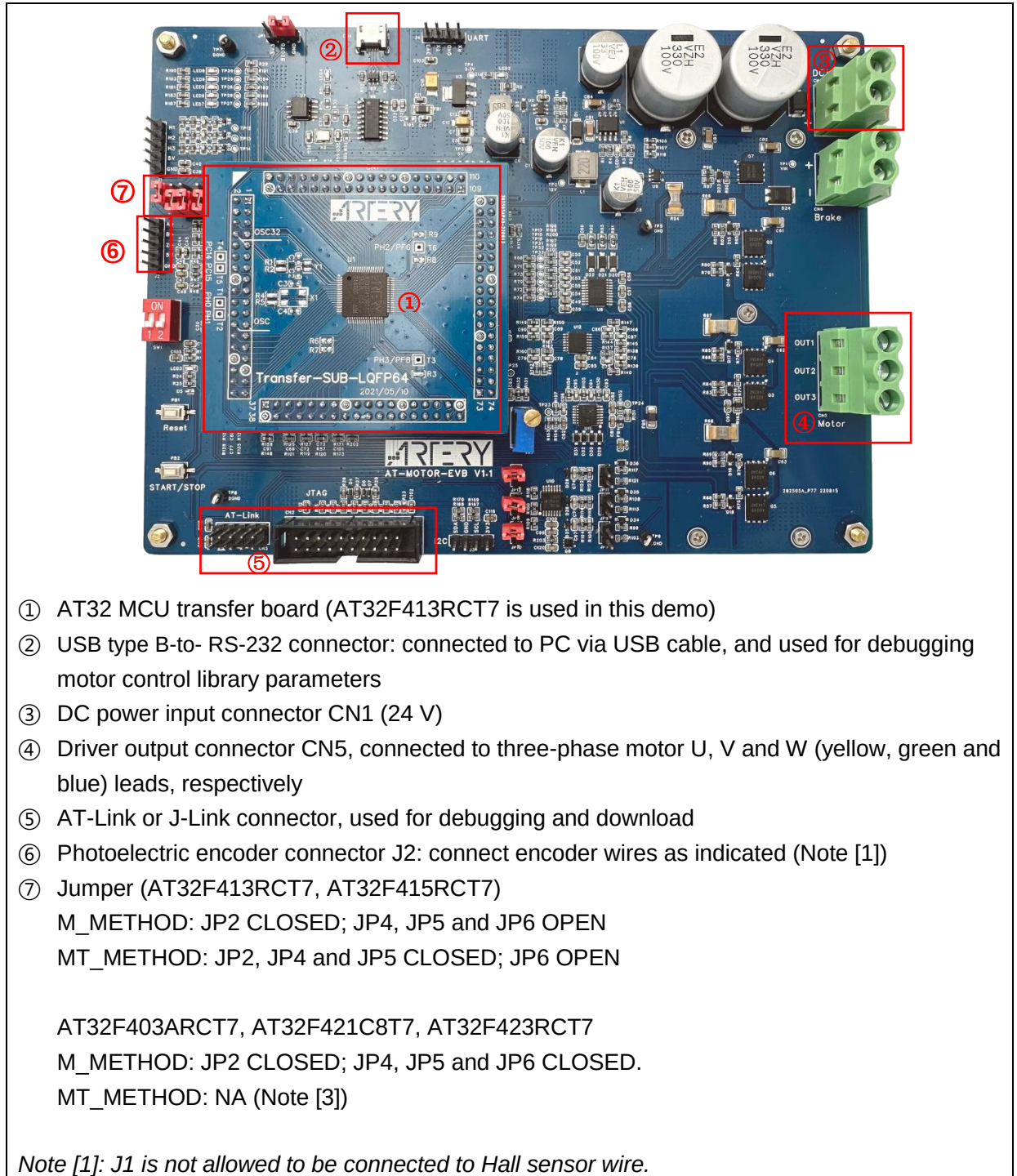
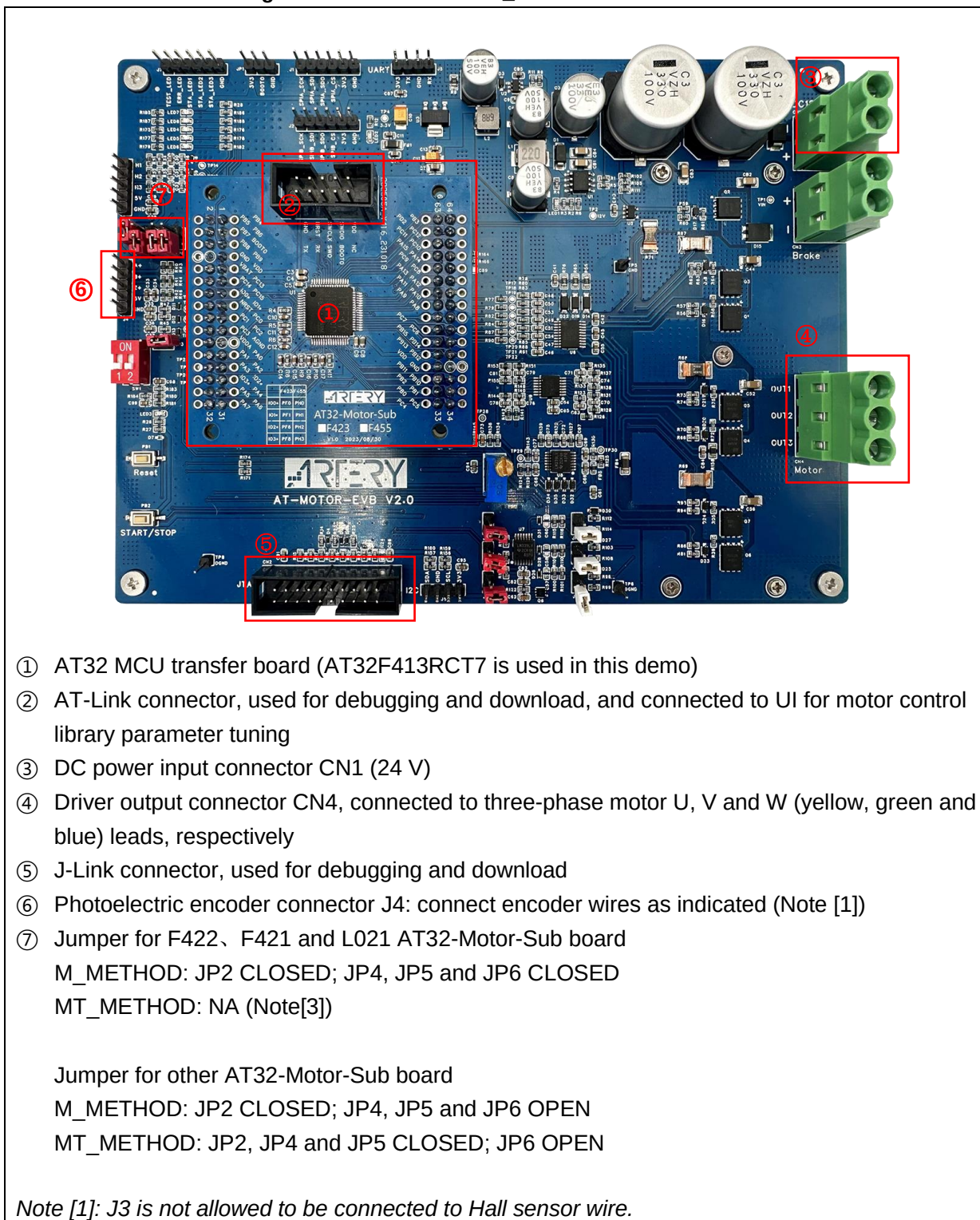


Figure 3. AT-MOTOR-EVB_V2.x evaluation board



2.2 Software requirement

- 1) Take the AT32F413 as an example. Open the demo project under AT32F413_MC_Library_Project\motor_evb_2v0\at32f413\pmsm_foc_incremental_encoder (Note [2]).
- 2) Run ArteryMotorMonitor.exe directly (installation not required).
- 3) In Keil IDE, go to **Options - Read/Only Memory Areas** to set ROM according to the Flash memory size of each AT32 MCU (see Table 2). For example, for AT32F413RCT7 with a 256KB Flash memory, its IROM1 start address is 0x8000000 with a size of 0x3F800, whereas its IROM2 start address is 0x803F800 with a size of 0x800 (see Figure 4). In AT32IDE, modify *ld file according to the Flash memory size of each AT32 MCU (see Figure 5). In IAR Systems, modify *icf file according to the Flash memory size of each AT32 MCU (see Figure 6).

Table 2. Correspondence between Flash size and ROM

Flash size	4032K	1024K	512K	448K	256K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x3EF000	0xFF800	0x7F800	0x6F000	0x3F800
IROM2(address)	0x83EF000	0x80FF800	0x807F800	0x0806F000	0x803F800
IROM2(size)	0x1000	0x800	0x800	0x1000	0x800

Flash size	128K	64K	32K	16K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x1FC00	0x0FC00	0x07C00	0x03C00
IROM2(address)	0x801FC00	0x800FC00	0x8007C00	0x8003C00
IROM2(size)	0x400	0x400	0x400	0x400

Note [2]: This demo project does not support Keil compiler version 5 and O0 optimization level for compiling simultaneously. Please use Keil compiler version 6 or O1 optimization level for compiling. In case of Keil_v5.33, since AT32 BSP source code does not support V6.15 compiler, the Keil compiler version 5 and O1 optimization level should be used.

Figure 4. AT32F413RCT7 ROM settings (Keil)

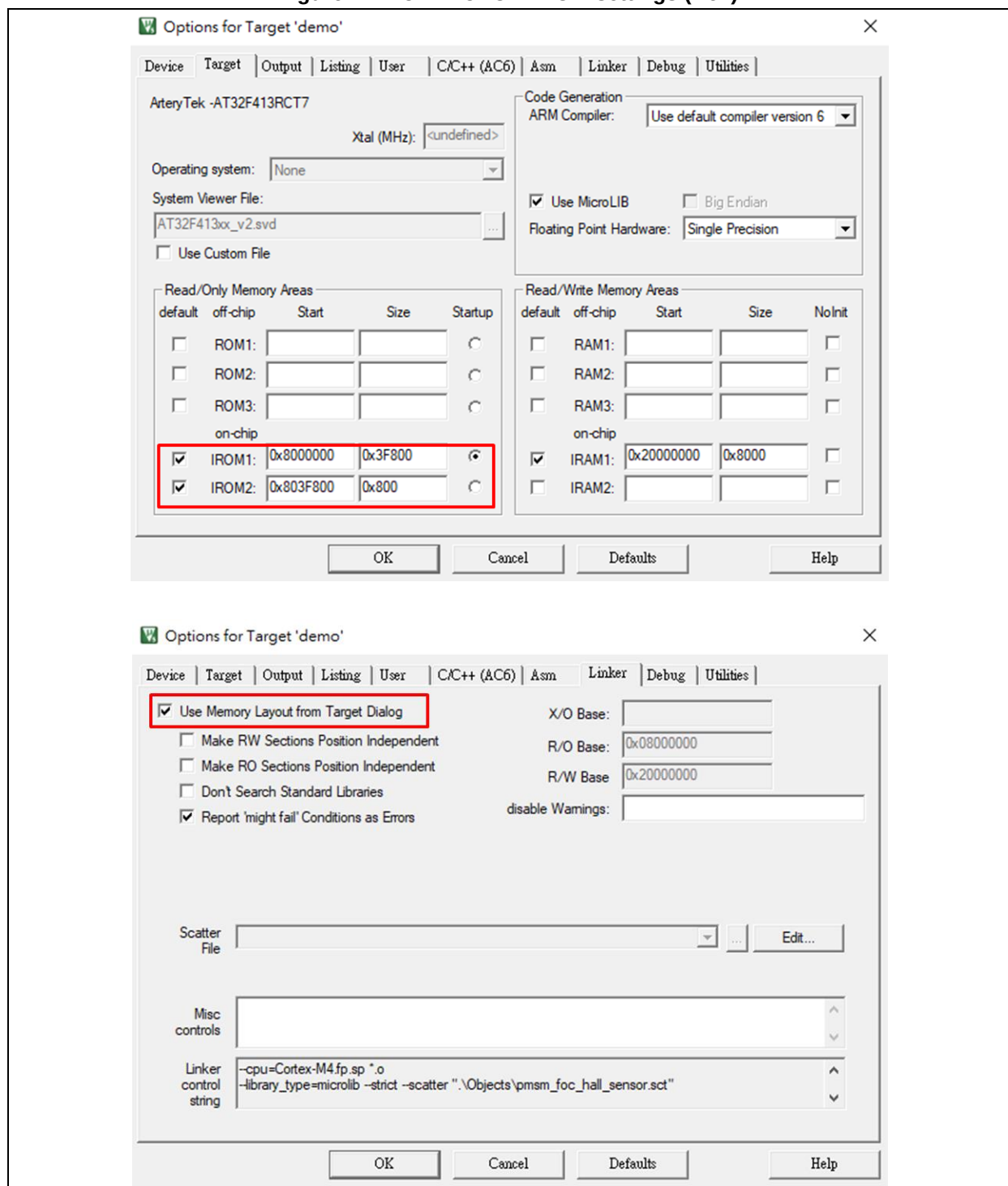


Figure 5. AT32F413RCT7 ROM settings (AT32IDE)

```

AT32F413xC_FLASH.ld X
25 _estack = 0x20008000; /* end of RAM */
26
27 /* Generate a link error if heap and stack don't fit into RAM */
28 _Min_Heap_Size = 0x200; /* required amount of heap */
29 _Min_Stack_Size = 0x400; /* required amount of stack */
30
31 /* Specify the memory areas */
32 MEMORY
33 {
34     FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 0x3F800
35     MC_DATA (r)      : ORIGIN = 0x0803F800, LENGTH = 0x800
36     RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 32K
37 }
38
39 /* Define output sections */
40 SECTIONS
41 {
42     /* The startup code goes first into FLASH */
43     .isr_vector :
44     {
45         . = ALIGN(4);
46         KEEP(*(.isr_vector)) /* Startup code */
47         . = ALIGN(4);
48     } >FLASH
49
50     .mc_data :
51     {
52         . = ALIGN(4);
53         . = ORIGIN(MC_DATA);
54         _MC_VectStoreAddr1 = .;
55         *(.MC_VectStoreAddr1);
56         . = ALIGN(4);
57         . = ORIGIN(MC_DATA) + 800;
58         _MC_VectStoreAddr2 = .;
59         *(.MC_VectStoreAddr2);
60         . = ALIGN(4);
61     } >MC_DATA
62
63     /* The program code and other data goes into FLASH */
64     .text :
65     {
66         . = ALIGN(4);
67         *(.text)           /* .text sections (code) */
68         *(.text*)          /* .text* sections (code) */
69         *(.glue_7)         /* glue arm to thumb code */
70         *(.glue_7t)        /* glue thumb to arm code */
71         *(.eh_frame)
72

```

Figure 6. AT32F413RCT7 ROM settings (IAR)

```

1  /****ICF**** Section handled by ICF editor, don't touch! ****/
2  /*-Editor annotation file-*/
3  /* IcfEditorFile="$TOOLKIT_DIR$\config\ide\IcfEditor\cortex_vl_0.xml" */
4  /*-Specials-*/
5  define symbol __ICFEDIT_intvec_start__ = 0x08000000;
6  /*-Memory Regions-*/
7  define symbol __ICFEDIT_region_ROM_start__ = 0x08000000;
8  define symbol __ICFEDIT_region_ROM_end__ = 0x0803F800 - 1;
9  define symbol __ICFEDIT_region_ROM1_start__ = 0x0803F800;
10 define symbol __ICFEDIT_region_ROM1_end__ = (__ICFEDIT_region_ROM1_start__ + 800 - 1);
11 define symbol __ICFEDIT_region_ROM2_start__ = (__ICFEDIT_region_ROM1_end__ + 1);
12 define symbol __ICFEDIT_region_ROM2_end__ = 0x0803F800 + 0x800 - 1;
13 define symbol __ICFEDIT_region_RAM_start__ = 0x20000000;
14 define symbol __ICFEDIT_region_RAM_end__ = 0x20007FFF;
15 /*-Sizes-*/
16 define symbol __ICFEDIT_size_cstack__ = 0x400;
17 define symbol __ICFEDIT_size_heap__ = 0x200;
18 /**** End of ICF editor section. ****ICF****/
19
20 define memory mem with size = 4G;
21 define region ROM_region = mem:[from __ICFEDIT_region_ROM_start__ to __ICFEDIT_region_ROM_end__];
22 define region ROM1_region = mem:[from __ICFEDIT_region_ROM1_start__ to __ICFEDIT_region_ROM1_end__];
23 define region ROM2_region = mem:[from __ICFEDIT_region_ROM2_start__ to __ICFEDIT_region_ROM2_end__];
24 define region RAM_region = mem:[from __ICFEDIT_region_RAM_start__ to __ICFEDIT_region_RAM_end__];
25
26 define block CSTACK with alignment = 8, size = __ICFEDIT_size_cstack__ { };
27 define block HEAP with alignment = 8, size = __ICFEDIT_size_heap__ { };
28
29 initialize by copy { readwrite };
30 do not initialize { section .noinit };
31
32 place at address mem:__ICFEDIT_intvec_start__ { readonly section .intvec };
33
34
35 place in ROM_region { readonly };
36 place in ROM1_region { readonly section .MC_VectStoreAddr1 };
37 place in ROM2_region { readonly section .MC_VectStoreAddr2 };
38 place in RAM_region { readwrite,
39 block CSTACK, block HEAP };

```

3 PMSM motor control library documents

Figure 7 shows the structure of motor control library documents and their relationships. The *motor_control_drive_param.h* file contains the motor control drive modes, motor parameters, control board parameters and controller parameters to be set by users. Users can set MCU peripheral parameters in *mc_hwio.h* file according to the connection relations between MCU peripherals and control board circuits. Relevant setting parameters are integrated in *mc_foc_globals.h* (*mc_bldc_globals.h*), and then their initial variables are set by functions in *mc_foc_globals.c* (*mc_bldc_globals.c*) and used by motor control library functions. For the MCU peripherals initialization, it is executed in *mc_hwio.c*.

Figure 7. Structure of motor control library documents

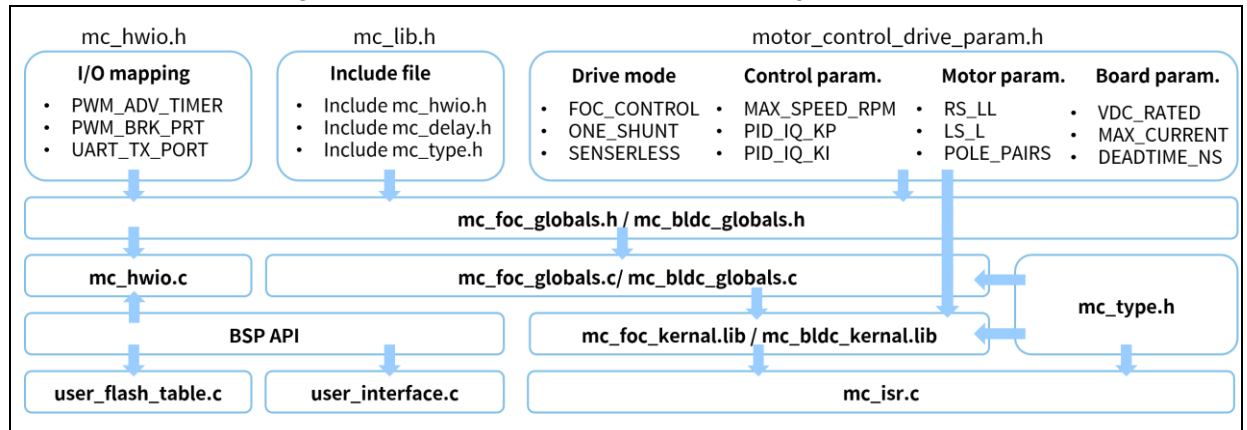


Figure 8 shows the document structure of the demo project. The “user” folder is user-defined and it contains the main program, peripheral program, writing parameters table into Flash, and parameter definition header files. The “firmware” folder contains BSP files, and the “mcilib” folder contains motor control library program, including delay functions, communication functions, global variable setting functions and motor control library functions. For details about motor control library functions, refer to AN0064 AT32_Motor_Control_Library_User_Guide.

Figure 8. Structure of motor control project

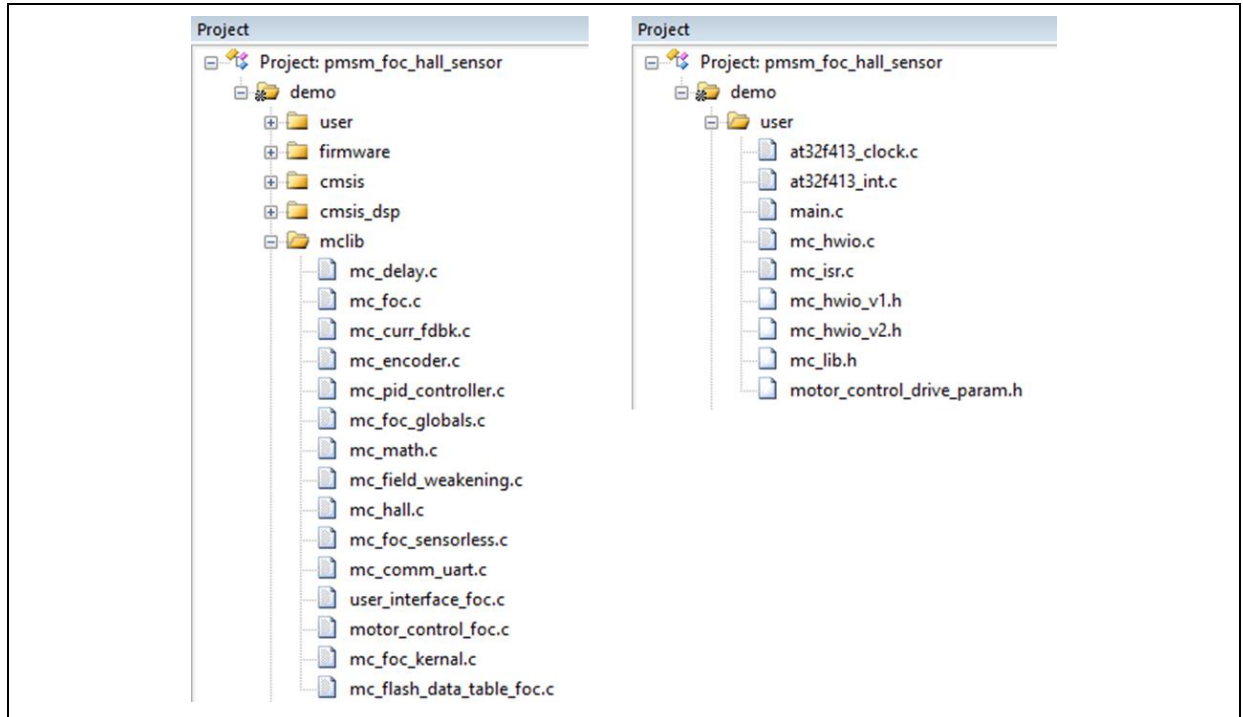


Table 3 lists motor control library documents.

Table 3. Description of motor control library documents

File name	Description
mc_lib.h	Unified header file management
motor_control_drive_param.h	Set motor drive modes (current sampling mode, sensor mode etc.), control parameters, driver parameters, motor parameters
mc_hwio.c	Hardware peripheral configuration
mc_hwio_v1.h	Hardware IO macro definitions (AT-MOTOR-EVB_V1.x)
mc_hwio_v2.h	Hardware IO macro definitions (AT-MOTOR-EVB_V2.x)
mc_isr.c	Relevant motor control interrupt functions
mc_flash_data_table_foc.c	Writing parameters table into Flash
mc_flash_data_table_foc.h	Relevant configuration for writing parameters table into Flash
mc_type.h	Global variables types and definitions, enumeration definition
mc_delay.c	Delay-related functions
mc_delay.h	Declaration of delay-related functions
mc_comm_uart.c	Configuration of peripherals related to communication interface
mc_comm_uart.h	Declaration and configuration of communication UART related functions
mc_pid_control.c	PID controller related functions
mc_pid_control.h	Declaration of PID controller related functions
mc_curr_fdbk.c	Current sensing related functions
mc_curr_fdbk.h	Declaration of current sensing related functions
mc_math.c	Filter related functions

mc_math.h	Declaration of filter related functions
mc_hall.c	Hall sensor related functions
mc_hall.h	Declaration of Hall sensor related functions
mc_foc_kernal.lib	Motor control library core functions (for Keil only) (applicable to M4F MCU)
mc_foc_kernal_noFPU.lib	Motor control library core functions (for Keil only) (applicable to M4 MCU)
mc_foc_kernal_m0plus.lib	Motor control library core functions (for Keil only) (applicable to M0+ MCU)
libmc_foc_kernal.a	Motor control library kernel functions (for AT32IDE only) (applicable to M4F MCU)
libmc_foc_kernal_noFPU.a	Motor control library kernel functions (for AT32IDE only) (applicable to M4 MCU)
libmc_foc_kernal_m0plus.a	Motor control library kernel functions (for AT32IDE only) (applicable to M0+ MCU)
mc_foc_kernal.a	Motor control library kernel functions (for IAR system only) (applicable to M4F MCU)
mc_foc_kernal_noFPU.a	Motor control library kernel functions (for IAR system only) (applicable to M4 MCU)
mc_foc_kernal_m0plus.a	Motor control library kernel functions (for IAR system only) (applicable to M0+ MCU)
mc_foc_kernal.h	Declaration of motor control library kernel functions
motor_control_foc.c	Motor control related functions
motor_control_foc.h	Declaration of motor control related functions
mc_foc.c	Vector control related functions
mc_foc.h	Declaration of vector control related functions
mc_encoder.c	Encoder related functions
mc_encoder.h	Declaration of encoder related functions
mc_field_weakening.c	Filed weakening related functions
mc_field_weakening.h	Declaration of filed weakening related functions
mc_foc_sensorless.c	Sensorless mode related functions
mc_foc_sensorless.h	Declaration of sensorless mode related functions
mc_curr_decoupling.c	Current decouple control related functions
mc_curr_decoupling.h	Declaration of current decouple control related functions
MTPA_MTPV_table.c	Maximum Torque Per Ampere (MTPA)/Maximum Torque Per Volt (MTPV) field-weakening lookup table function
MTPA_MTPV_table.h	Declaration of Maximum Torque Per Ampere (MTPA)/Maximum Torque Per Volt (MTPV) field-weakening lookup table function
mc_foc_globals.c	Global variables definition and default values, global functions definition
mc_foc_globals.h	Global variables, global functions definition, macro definition
user_interface_foc.c	Functions related to communication interface
user_interface_foc.h	Declaration of functions related to communication interface

1) *motor_control_drive_param.h* file

- This file is divided into four parts, which are used to configure drive mode, control parameters, motor parameters and board parameters.
- Table 4 lists the definitions of modes that can be set according to the hardware and motor configuration. In this demo project, the photoelectric incremental encoder is used. Users can configure AB or ABZ according to the hardware configuration, and select MT_METHOD or M_METHOD for speed detection (Note [3]). The current sampling mode (three-shunt, two-shunt or single-shunt current sampling mode) is set according to hardware configuration. In addition, the field weakening control is optional.

Table 4. Drive mode definitions

Definition item	Description
FOC_CONTROL	FOC vector control mode
THREE_SHUNT	Three-shunt current sampling
TWO_SHUNT	Two-shunt current sampling
ONE_SHUNT	Single-shunt current sampling
U_V_SHUNT	U & V shunt (for TWO_SHUNT only)
V_W_SHUNT	V & W shunt (TWO_SHUNT only)
U_W_SHUNT	U & W shunt (TWO_SHUNT only)
AT_MOTOR_EVB_V1	Motor control board V1.x (AT_MOTOR_EVB_V1.x)
AT_MOTOR_EVB_V2	Motor control board V2.x (AT_MOTOR_EVB_V2.x)
GATE_DRIVER_LOW_SIDE_INVERT	Enable low-side MOS inverted output
INCREM_ENCODER	Photoelectric incremental encoder
REVERSE_ENCODER_COUNT	Reverse counting (Photoelectric incremental encoder)
ABZ	AB signal with zero position (photoelectric incremental encoder)
AB	AB signal without zero position (photoelectric incremental encoder)
M_METHOD	M-method with speed measurement (photoelectric incremental encoder) (Note [3])
MT_METHOD	M/T method with speed measurement (photoelectric incremental encoder) (Note [3])
FIELD_WEAKENING	Field weakening control
CURRENT_LP_FILTER	Get d/q axis current low-pass filter signals (vector control)
AC_CURRENT_LP_FILTER	Get A/B/C Phase Low-Pass Filtered Current Signals (Vector Control/FOC)
INTERNAL_CLOCK_SOURCE	Use MCU internal oscillator as a clock source
MOTOR_PARAM_IDENTIFY	Motor winding parameters identification
CURRENT_DECOUPLE_CTRL	Current decouple control (AT32L021 series Excluded)
TORQUE_CTRL_WITH_SPEED_LIMIT	Torque (current) control, including maximum speed limitation

BRAKING_RESISTOR	Use an external braking resistor
SPMSM	Surface-mounted permanent magnet synchronous motor
IPMSM	Interior Permanent Magnet Synchronous Motor
IPM_MTPA_MTPV_TABLE	MTPA/MTPV Table Generation (IPMSM-Specific) (AT32L021 series Excluded)
IPM_MTPA_MTPV_CTRL	MTPA/MTPV lookup table control (IPMSM-Specific) (AT32L021 series Excluded)
USE_MOTOR_MONITOR	Artery motor control UI tool
MOS_RDS_SHUNT	Current Sampling Utilizing Low-Side MOSFET On-Resistance
TWO_ADC_CONVERTERS	Current Sampling with Dual-ADC Synchronization (Applicable for MCUs with dual or more ADCs; Not compatible with ONE_SHUNT topology)

Note [3]: *M_METHOD* and *MT_METHOD* are two methods for speed measurement of encoder. Generally, *M_METHOD* is set for speed detection, which requires capturing the pulse number of encoder only. For more accurate speed detection, it is recommended to set *MT_METHOD*, which requires capturing the pulse number of encoder and an additional set of timers to capture the encoder pulse duration. Set JP4 and JP5 CLOSED for AT-MOTOR-EVB hardware circuit to configure *MT_METHOD* by an additional set of timer.

- Table 5 lists the definitions of control parameters that can be defined according to the hardware, motor specifications and control characteristics. Some parameters can be set by the UI tuning, such as PI controllers of d-axis and q-axis currents, speed PI controller, etc.

Table 5. Control parameter definitions

Definition item	Description
PWM_FREQ	PWM output frequency (unit: Hz)
MOTOR_CONTROL_MODE	Motor control mode (speed control, torque control, etc.; refer to motor_control_mode in type.h for details)
CTRL_SOURCE	Control source settings (external source control/software control)
UI_UART_BAUDRATE	Baud rate of UI communication serial port
TUNE_TARGET_CURRENT	Tune target current value for current loop PI control parameters (unit: ampere)
TUNE_CURRENT_TOTAL_PERIOD	Tune total period of current pulse for current loop PI control parameters (unit: ms)
TUNE_CURRENT_STEP_PERIOD	Tune current pulse step period for current loop PI control parameters (unit: ms)
SPEED_LOOP_FREQ	Speed loop frequency (unit: Hz)
MIN_SPEED_RPM	Minimum motor speed (unit: rpm)
MAX_SPEED_RPM	Maximum motor speed (unit: rpm)

Definition item	Description
MIN_POSCTL_SPD	Minimum speed for position loop control (unit: rpm)
ACC_SPD_SLOPE	Slope of acceleration (unit: rpm/ms, systick =1kHz)
DEC_SPD_SLOPE	Slope of deceleration (unit: rpm/ms, systick =1kHz)
SP_MAX_VOLT	Maximum voltage of external command source (unit: voltage)
SP_THRESHOLD	Threshold voltage of external command source (unit: voltage)
SP_RUN_VALUE	Minimum voltage to start running in external source mode (unit: voltage)
SP_STOP_VALUE	Maximum voltage to stop running in external source mode (unit: voltage)
PID_SPD_KP_DIV	Speed control proportional gain divisor (Q16 mode)
PID_SPD_KI_DIV	Speed control integral gain divisor (Q16 mode)
PID_SPD_KP_DEFAULT	Speed control proportional gain (Q15 mode)
PID_SPD_KI_DEFAULT	Speed control integral gain (Q15 mode)
POSITION_LOOP_FREQ	Position loop control frequency (unit: Hz)
MAX_POSITION_ANGLE	Maximum position angle (unit: Degree)
MIN_POSITION_ANGLE	Minimum position angle (unit: Degree)
CMD_TO_VAL_GAP	Define the gap value between rotor position and its target to use PID_POS_KI_DEFAULT_STABLE
SMALL_POS_CMD_GAP	When the rotor position difference is within a certain range of the target position, the minimum speed command (MIN_POSCTL_SPD) is used for position loop control
PID_POS_KP_DEFAULT	Rotor position control proportional gain (Q15 mode)
PID_POS_KI_DEFAULT	Rotor position control integral gain (Q15 mode)
PID_POS_KI_DEFAULT_STABLE	Rotor position control integral gain (Q15 mode) (rotor position is close to target position)
PID_POS_KD_DEFAULT	Rotor position control derivative gain (Q15 mode)
PID_POS_KP_GAIN_DIV	Rotor position control proportional gain divisor (Q16 mode)
PID_POS_KI_GAIN_DIV	Rotor position control integral gain divisor (Q16 mode)
PID_POS_KD_GAIN_DIV	Rotor position control derivative gain divisor (Q16 mode)
PID_ID_KP_DEFAULT	d axis current control proportional gain (Q15 mode)
PID_ID_KI_DEFAULT	d axis current control integral gain (Q15 mode)
PID_ID_KP_GAIN_DIV	d axis current control proportional gain divisor (Q16 mode)
PID_ID_KI_GAIN_DIV	d axis current control integral gain divisor (Q16 mode)
PID_IQ_KP_DEFAULT	q axis current control proportional gain (Q15 mode)
PID_IQ_KI_DEFAULT	q axis current control integral gain (Q15 mode)
PID_IQ_KP_GAIN_DIV	q axis current control proportional gain divisor (Q16 mode)
PID_IQ_KI_GAIN_DIV	q axis current control integral gain divisor (Q16 mode)

Definition item	Description
AUTO_TUNE_CURR_BANDWIDTH	Current PI controller bandwidth (used for self-tuning of current PI controller parameters)
CURR_LP_BANDWIDTH	d/q-axis current low-pass filter band width (unit: Hz)
AC_CURR_LP_BANDWIDTH	a/b/c phase current low-pass filter band width (unit: Hz)
OLC_ANGLE_INC	Open loop control angle incremental (at each PWM output frequency)
OLC_VOLT	Open loop control voltage output (unit: voltage)
ALIGN_VOLT	Encoder aligned voltage (unit: voltage)
ENC_OFFSET	Encoder aligned offset (unit: resolution or count per revolution)
FW_MAX_ID_CURR	Maximum d-axis current in field weakening control (unit: ampere)
FW_KP_GAIN	Field weakening control proportional gain (Q15 mode)
FW_KI_GAIN	Field weakening control integral gain (Q15 mode)
FW_KP_GAIN_DIV	Field weakening control proportional gain divisor (Q16 mode)
FW_KI_GAIN_DIV	Field weakening control integral gain divisor (Q16 mode)
HIGH_PWM_DUTY_CYCLE	Dynamic Adjustment of Current Sampling Position at High PWM Duty Cycle (TWO_ADC_CONVERTERS only)
MAX_DUTY_PCT	Maximum PWM Duty Cycle Setting

- Table 6 lists drive-related parameter definitions, such as dead time, shunt resistor and current amplifier gain. The corresponding parameters should be modified if other motor drive board is used.

Table 6. Driver parameter definitions

Definition item	Description
VDC_RATED	DC-BUS voltage(unit: voltage)
V_SENSE_GAIN	Voltage feedback ratio
ADC_REFERENCE_VOLT	ADC reference voltage (unit: voltage)
ADC_DIGITAL_SCALE_12BITS	ADC resolution
SYSTEM_CORE_CLOCK	System frequency (unit: Hz)
TMR_CLK	Timer clock frequency (unit: Hz)
DEADTIME_CLK_SFT_BITS	Shift bit number of frequency divider for dead time generator
DEADTIME_NS	Dead time (unit: ns)
MIN_INTERVAL_TIME	Minimum interval time between two-phase signals during PWM shift (unit: ns)
ADC_TRIG_DELAY_TIME	Current sampling delay time (unit: us)
ADC_TRIG_LAG_TIME	ADC trigger lag time for current sensing (Two-Phase Low-Level Condition) (Unit: ns) (only for TWO_ADC_CONVERTERS &&

Definition item	Description
	THREE_SHUNT)
SW_OP_INP_MODE_LEAD_TIME	OP Input Mode Switching Lead Time Prior to ADC Trigger (for chips with internal OP like M412)
MAX_CURRENT	Maximum peak phase current of motor (unit: ampere)
MIN_CURRENT	Minimum peak phase current of motor (unit: ampere)
CURRENT_SPAN_SHIFT	Number of shifts for current per-unit normalization
R_SHUNT	Shunt resistance (unit: Ω)
OP_GAIN	Current amplifier gain
CURR_OFFSET_VOLT	Zero current offset (unit: voltage)
OVER_CURRENT_SW	Software Overcurrent Threshold (unit: ampere)
DAC_VREF_SOURCE	DAC Source Selection for Internal OP Overcurrent Protection (Dedicated to chips with internal OP, e.g., M412 series)
OCP_CURRENT	Hardware Overcurrent Threshold (unit: ampere)
BUS_CURR_CMP_OCP_VOLT	Internal OP Overcurrent Protection Voltage (unit: volt) (Dedicated to chips with internal operational amplifier, e.g., M412 series)
TMR_BRK_FILTER_COUNT	Overcurrent Protection Break Input Filter Count (Dedicated to chips with break input function, e.g., M412 series)
OVER_VOLT_THRESHOLD	Over-voltage threshold point (unit: voltage)
UNDER_VOLT_THRESHOLD	Under-voltage threshold point (unit: voltage)
V0_V	Parameter V0 of the temperature and voltage linear approximation function of NTC voltage divider circuit (Note [4])
T0_C	Parameter T0 of the temperature and voltage linear approximation function of NTC voltage divider circuit (Note [4])
dV_dT	Parameter dV/dT of the temperature and voltage linear approximation function of NTC voltage divider circuit (Note [4])
OVER_TEMP_THRESHOLD	Over-temperature threshold point (unit: Celsius degrees)
MC_ERROR_MASK	Error detection mask

Note [4]: Approximate curve equation of voltage-temperature relation is $V[V] = V0 + dV/dT[V/Celsius] * (T - T0)[Celsius]$.

- Table 7 lists definitions of motor parameters, including number of pole-pairs, encoder parameters and Hall sensor parameters.

Table 7. Motor parameter definitions

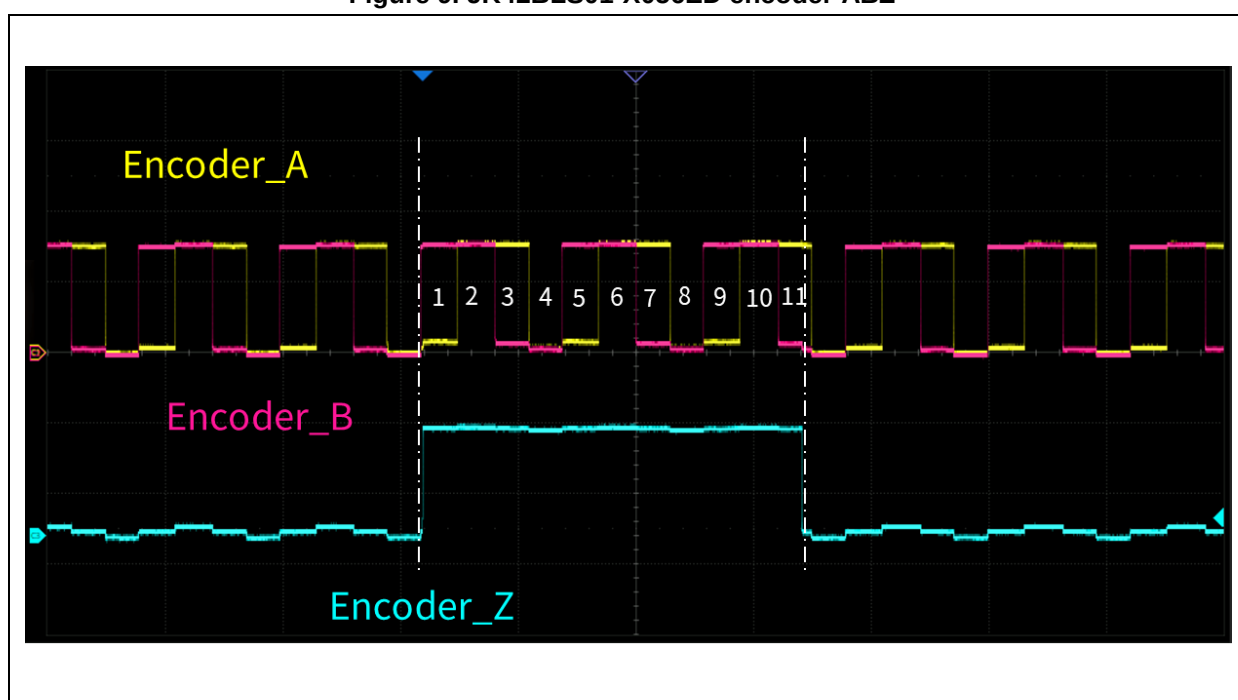
Definition item	Description
POLE_PAIRS	Number of pole-pairs
KE	Back EMF constant (KE) (unit: V/rpm)

Definition item	Description
LD_LQ_RATIO	Ld to Lq ratio
NOMINAL_CURRENT	Motor nominal current (unit: ampere)
ENCODER_PPR	Pulses per revolution of encoder (unit: pulse per revolution)
ENC_IDX_COUNT	Encoder index signal width (unit: count) (Note[5])
ENC_STALL_TIME	Encoder stall time (unit: ms)

Note [5]: It is suitable for photoelectric incremental encoder with ABZ signal.

Figure 9 shows ABZ signal of JK42BLS01-X056ED encoder. The number of the width (from Z signal rising edge to falling edge) that is aligned with AB signal is counted 11; therefore, the ENC_IDX_COUNT is set to 11 (which is counted 1 or 2 for general photoelectric encoder).

Figure 9. JK42BLS01-X056ED encoder ABZ



2) *mc_hwio.h* file

- This file is mainly used for configuring definitions based on the mapping of user hardware IO interface and MCU peripherals, and it also includes function declarations in *mc_hwio.c* file. Table 8 lists the configurations of peripherals such as ADC, TMR, USART and EXINT, interrupt functions and the corresponding DMA channels.

Table 8. Configurations of peripherals (AT32F413RCT7)

Peripheral	Interrupt function	DMA channel	Description
ADC1	N/A	DMA1_CH1	ADC1 ordinary channel conversion (BUS voltage, MOS temperature, external speed/torque command, three-phase BEMF voltage and BUS

			current)
ADC1	ADC1_2_IRQn	N/A	ADC1 preempted channel conversion (phase current or bus current)
TMR1	TMR1_OVF_TMR10_IRQn	N/A	FOC control interrupt
TMR1	TMR1_BRK_TMR9_IRQn	N/A	PWM output disable
TMR1_CH4	N/A	DMA1_CH4	Set sampling time point in one-shunt mode
TMR5	N/A	N/A	Encoder count mode
TMR3	TMR3_GLOBAL_IRQHandler	N/A	Encoder capture mode
TMR11	TMR1_TRG_HALL_TMR11_IRQHandler	N/A	Speed control loop interrupt
USART1_TX	N/A	DMA1_CH2	USART1 TX data
USART1_RX	N/A	DMA1_CH3	USART1 RX data
EXINT	EXINT4_IRQn	N/A	Encoder zero position calibration
EXINT	EXINT15_10_IRQn	N/A	START/STOP button (start/stop motor)

3) *mc_hwio.c* file

- This file is mainly configured the MCU peripherals based on user hardware, such as TMR, ADC, DMA and GPIO. It also includes some basic control functions such as button, LED, DIP switch. Refer to Table 9 for details.

Table 9. Peripheral configuration related functions

Function	Description
<code>nvic_config</code>	Interrupt priority configuration
<code>tmr_pwm_init</code>	PWM timer, crm clock, GPIO and DMA configuration
<code>encoder_time_init</code>	Incremental photoelectric encoder timer, crm clock, GPIO and EXINT configuration
<code>encoder_capture_timer_init</code>	Incremental photoelectric encoder capture timer, crm clock and GPIO configuration (dedicated for FOC)
<code>adc_ordinary_config</code>	ADC ordinary channel ADC, DMA and GPIO configuration
<code>adc_preempt_config</code>	ADC preempted channel ADC, DMA and GPIO configuration
<code>speed_timer_init</code>	Speed control timer and crm clock configuration
<code>uart_init</code>	UART crm clock, GPIO and UART configuration
<code>button_exint_init</code>	Button interrupt event EXINT configuration
<code>led_config</code>	LED GPIO initialization and crm clock configuration
<code>led_on</code>	LED ON
<code>led_off</code>	LED OFF
<code>led_toggle</code>	LED toggle (from ON to OFF, or from OFF to ON)
<code>led_init</code>	LED initialization configuration
<code>led_blink</code>	LED blinks
<code>mode_switch_init</code>	DIP switch configuration
<code>motor_parameter_ID_config</code>	Set TMR and ADC relating to motor winding parameter identification
<code>hwdiv_config</code>	Hardware divider configuration (for AT32L021 only)
<code>get_int_vref_cal_ratio</code>	Detect VDPA reference voltage ratio, used for ADC value calibration
<code>opa_init</code>	Busbar Current Amplification OP Configuration (AT32M412 and other chips with internal OP only)
<code>opa_calibration</code>	Busbar Current Amplification OP Calibration (AT32M412 and other chips with internal OP only)
<code>ocp_cmp_config</code>	Busbar Current Protection CMP Configuration (AT32M412 and other chips with internal CMP only)

4) *mc_isr.c* file

- This file includes interrupt handler functions for motor control program, such as ADC, TMR and SYSTICK interrupts. Refer to Table 10 for details.

Table 10. Motor control related interrupt functions (AT32F413RCT7)

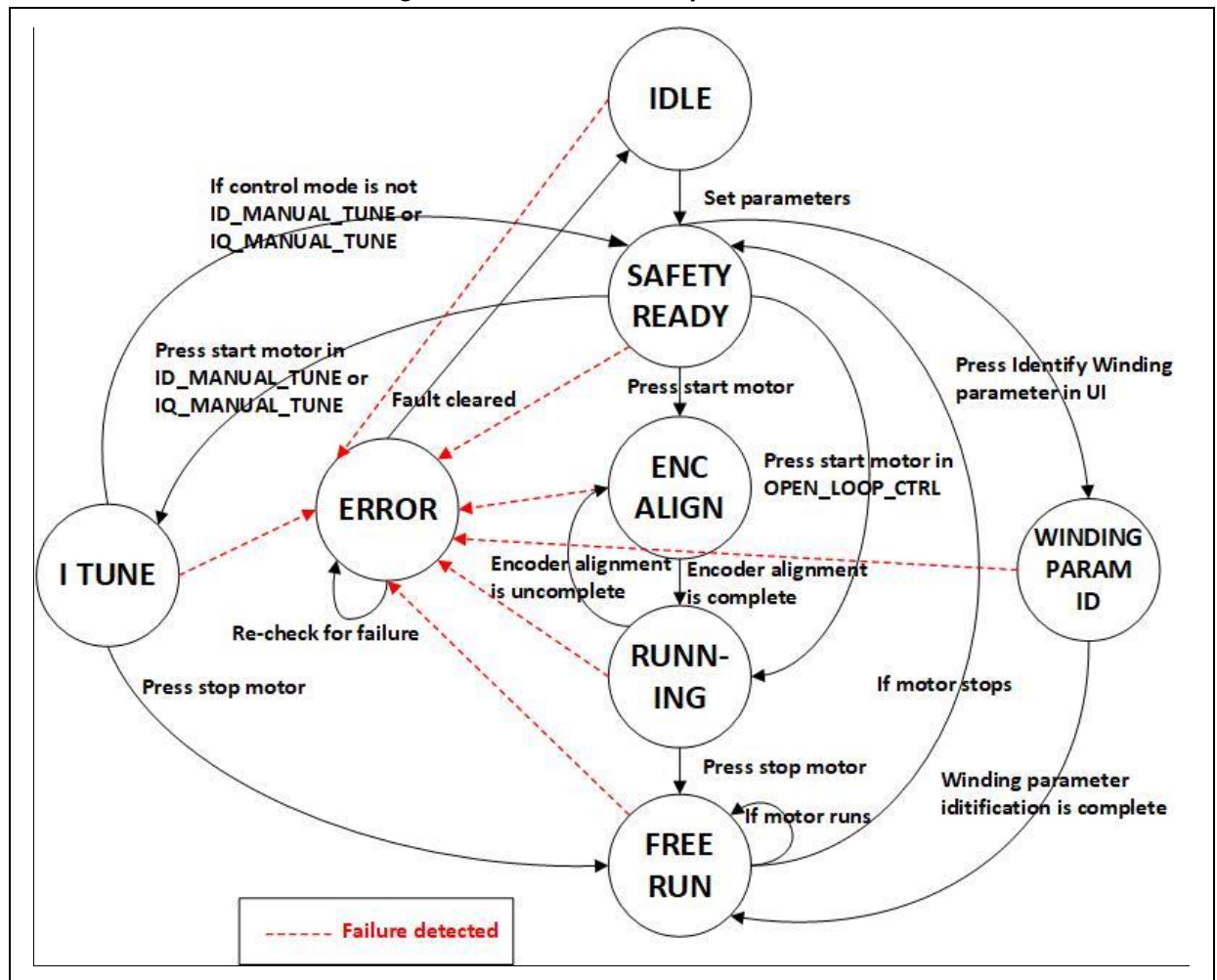
Function	Description
ADVTMR_PWM_CYCLE_IRQ (TMR1_OVF_TMR10_IRQHandler)	TIMER1 update interrupt function: get rotor angle, vector control, open loop control, voltage loop control and current loop control
ADVTMR_PWM_BRK_IRQ (TMR1_BRK_TMR9_IRQHandler)	TMR1 brake input interrupt function: PWM output disable
ADC_SHUNT_SAMP_READY_IRQ (ADC1_2_IRQHandler)	ADC1 interrupt function: current sampling complete
ENCODER_CAPTURE_IRQ (TMR3_GLOBAL_IRQHandler)	Encoder capture interrupt function
EXINT_ENCODER_IDX_IRQ (EXINT4_IRQHandler)	External IO interrupt function: encoder zero position calibration
SPEED_LOOP_TIMER_IRQ (TMR1_TRG_HALL_TMR11_IRQHandler)	Speed control loop interrupt function and get rotor speed
SysTick_Handler	System interrupt function (1ms), includes state machine program
BUTTON_EXINT_IRQ (EXINT15_10_IRQHandler)	External IO interrupt function: START/STOP button (start/stop motor)

4 State machine

4.1 Description of state

Figure 10 shows the state machine of the application example of motor control library. The initial settings of each state are constantly checked by the Main program; when the state changes, the initial settings are set for the new state. In addition, the state machine runs in the SysTick Handler interrupt function every minisecond to ensure that it is checked in real time. Each state is checked for fault; once detected, the state machine enters “Error” state and stop driving to avoid damage to the motor or motor drive board, and the motor stops until the fault is cleared.

Figure 10. State machine process flow



It includes Idle, Safety ready, Running, Free run, I_tune, Enc_align and Error, as detailed below.

Idle

It is the initial state of state machine, and motor idles in this state. The state machine returns to “Idle” state when the error condition is released.

Safety ready

This state indicates that the motor can be started up safely after all parameters are set and checked and the current offset is obtained in the “Idle” state.

Running

The motor is running in this state. Through the UI software, users can on-line adjust parameters (target speed, target current, etc.) or stop the motor.

Free run

In this state, the drive stops voltage output, and the motor speed will slowly down to zero. Program remains in this state until the motor stops completely, and then returns to the “Safety ready” state.

I_tune

This state is used for adjusting PID controller parameters. In this state, users can adjust the target current, current loop KP and KI values through UI software. A step current is generated in this state, based on which parameters are adjusted.

Enc_align

This state is for the zero calibration of the encoder. After the driver is restarted, the zero calibration will be performed automatically before starting the motor, or user can start the zero calibration function on the UI software.

Winding param Id

This refers to the state of motor winding parameter identification. Users can enable this feature via UI interface. The winding parameter identification feature is often used for tuning a new motor. The tuned parameters can be used for sensorless vector control and current PI controller parameters self-tuning.

Error

Jump to this state in case of any error.

Table 11 lists each state in details and state switching conditions.

Table 11. Initial state and state switching conditions

Initial state	End state	State switching conditions
IDLE	SAFETY REDAY	The control source is the UI tool, and initial parameters are set.
		The control source is the motor control board, POTENTIOMETER command is given less than the default, and initial parameters are set.
SAFETY REDAY	ENC ALIGN	When the control mode is not ID_MANUAL_TUNE or IQ_MANUAL_TUNE, click the START MOTOR (UI tool) or START/STOP button on the motor control board.
	I TUNE	When the control mode is ID_MANUAL_TUNE or IQ_MANUAL_TUNE click START MOTOR (UI tool) or START/STOP button on the motor control board.
	RUNNING	When the control mode is OPEN_LOOP_CTRL, click START MOTOR (UI tool) or START/STOP button on the motor control board.
	WINDING_PARAM_ID	Click Identify Winding parameter (UI tool).
ENC ALIGN	RUNNING	When the control mode is not OPEN_LOOP_CTRL, and encoder align is completed.
RUNNING	ENC ALIGN	When the control mode is not OPEN_LOOP_CTRL, and encoder align is not completed.
	FREE_RUN	Click STOP MOTOR (UI tool) or START/STOP button on the motor control board.

FREE_RUN	SAFETY REDAY	Motor stops.
	FREE_RUN	Motor runs.
I TUNE	SAFETY REDAY	The control mode is not ID_MANUAL_TUNE or IQ_MANUAL_TUNE.
	FREE_RUN	Click STOP MOTOR (UI tool) or START/STOP button on the motor control board.
WINDING_PARAM_ID	FREE_RUN	Motor winding parameter identification is completed.
ERROR	IDLE	Fault is cleared.

5 How to use demo project

For details, refer to *AN0217_AT32_PMSM_FOC_Incremental_Encoder*.

6 Revision history

Table 12. Document revision history

Date	Version	Revision note
2023.01.13	2.0.0	Initial release.
2023.04.20	2.0.1	Added positioning control feature, and updated operation instruction of the latest version of UI program.
2023.10.03	2.0.2	Updated description of motor control library documents, and added applicable products in the first page.
2024.02.27	2.0.3	Added AT-MOTOR-EVB V2.0 wiring descriptions, macro definitions related to auto identification of motor winding parameters and self-tuning of current PI controller parameters, updated description of state machine, and added instructions to motor parameters adjustment.
2024.05.24	2.1.0	Updated tuning procedures and descriptions.
2024.12.27	2.1.1	Added applicable products, including AT32F435/437 and AT32L021 series; Added IAR IDE and the corresponding Flash ROM configuration; Updated some definitions and description.
2025.04.11	2.1.2	Added Support for AT32F425 series and MTPA/MTPV-related macro definitions
2025.10.09	2.1.3	Added new supported models, fixed flash configuration table and parameter updates

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.